

Talk to Me, Jarvis: An Open-Source Edge-Deployable Voice Assistant Framework for Autonomous Racecars

Daniel Henel, Frederik Werner, Alexander Langmann, Johannes Betz

Abstract—Recent advances in large language models have improved their effectiveness as back-end components for voice assistants, particularly in intent understanding and context-aware input classification. However, online-hosted models introduce network dependency and variable inference latency, limiting their suitability for time-critical autonomous driving applications.

In this work, we address these issues by developing Jarvis, an offline voice assistant for high-level behavioral commands of autonomous vehicles. Its architecture integrates speech recognition and synthesis with natural language command classification into a lightweight, local framework.

Jarvis’ core component is a text-to-command classifier, built using a domain-specific fine-tuning of the Mistral 7B model, demonstrating low-latency inference. Our experimental evaluation demonstrates that our solution outperforms larger online-hosted models, achieving 97.63 % intent recognition accuracy with an average processing latency of 1.39 s, making it well-suited for operations requiring quick response times. To support further research and fine-tuning, we provide an open-source implementation.

Index Terms—voice assistant, autonomous driving, intent recognition, large language model

I. INTRODUCTION

In racing, fractions of a second can decide between victory and defeat. Autonomous racing vehicles must make split-second decisions to outperform competitors, placing high demands on their software systems [1]. This requirement extends to the AI models that support these vehicles, which must be optimized to ensure both speed and efficiency while maintaining reliable performance.

Despite a high degree of autonomy, human input is still needed in certain situations in autonomous racing, e.g., issuing high-level commands like start or stop, or reacting to unexpected situations on the track that require sending the car to the pit lane. Traditional interfaces for these commands are usually provided through a graphical user interface (GUI) or a command line, which require visual attention and increase reaction time.

To address this problem, voice assistants provide an alternative solution, allowing the human operator to issue commands without looking away from the live telemetry. However, developing such voice assistants is not

D. Henel, A. Langmann and J. Betz are with the Professorship of Autonomous Vehicle Systems, TUM School of Engineering and Design, Technical University of Munich, 85748 Garching, Germany; Munich Institute of Robotics and Machine Intelligence (MIRMI)

F. Werner is with the Institute of Automotive Technology, TUM School of Engineering and Design, Technical University of Munich, 85748 Garching, Germany; Munich Institute of Robotics and Machine Intelligence (MIRMI). This work has been accepted for publication in the Proceedings of the IEEE Intelligent Transportation Systems Conference (ITSC 2026)

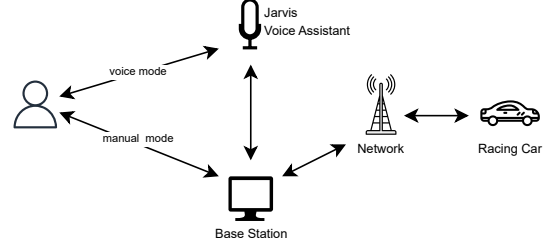


Fig. 1. System Architecture: Jarvis is a voice assistant for human operators used to control a autonomous racecar via voice commands.

straightforward. Challenges like latency, command detection reliability, and network dependency need to be addressed to design a useful voice assistant system.

Another key challenge arises from the inherent mismatch between the unbounded space of natural language expressions and the limited set of executable commands. For example, an operator may issue a “stop” command in many different ways - “halt the car”, “bring it to a stop”, or simply “stop” - all of which must be correctly recognized and mapped to the same underlying action. This variability motivates the use of Large-Language-Models (LLMs) to capture semantic equivalence across phrasings.

A. Contributions

This work makes the following contributions:

- A comparison of online-hosted and locally deployed language models for voice command classification, evaluating both inference latency and intent recognition accuracy.
- A fine-tuning and deployment pipeline for a lightweight local model that achieves superior intent recognition accuracy while reducing inference latency compared to online alternatives.
- An open-source, offline voice assistant architecture integrating wake-word detection, speech transcription, and command classification, demonstrated on resource-constrained mobile hardware (github.com/TUM-AVS/jarvis_voice_assistant).

B. Related Work

LLMs are increasingly used in various components of autonomous driving, such as scene understanding and multimodal perception [2], [3], planning and decision making [4], [5], explainability and human-machine

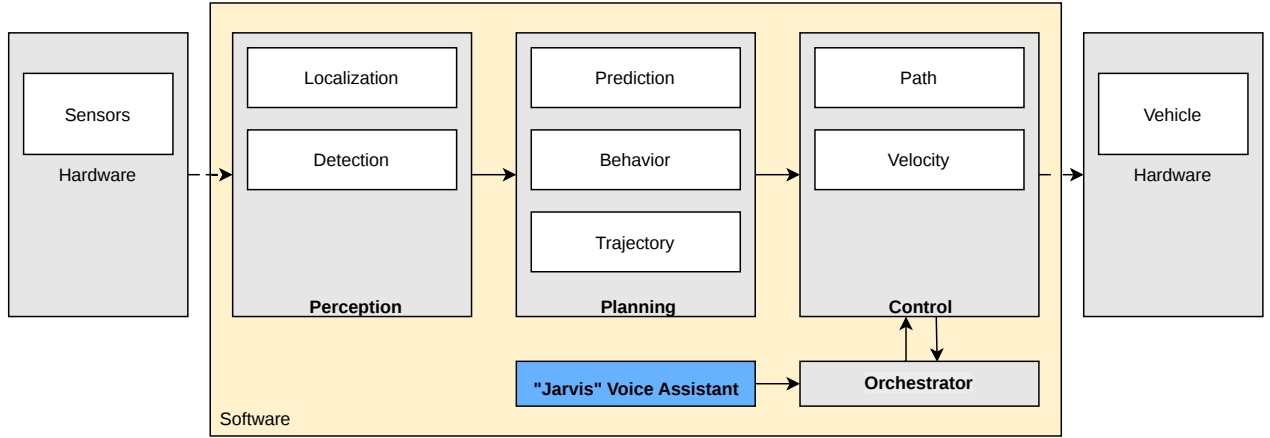


Fig. 2. Autonomous driving software modules.

interaction [6], [7], commonsense and traffic rule reasoning [8], [9].

Many researchers investigate the use of LLMs in intent recognition and command classification [10]–[12], focusing on translating natural language or contextual cues into actionable driving behaviors. Cui et al. [13] propose LLMs for Autonomous Driving (LLM4AD), which conceptualize the LLM as a cognitive brain for autonomous vehicles, capable of reasoning and intent understanding. By leveraging the broad knowledge of large-scale models like GPT-4 [14], LLM4AD aims to interpret abstract human inputs, such as adjusting driving style e.g. because a passenger is in a hurry, and translate them into high-level control strategies. However, while LLM4AD provides a benchmark for general passenger comfort, it relies on cloud-based Application Programming Interfaces (APIs) that introduce notable inference latencies and depend on a stable network connection. Furthermore, the framework is primarily designed for civilian vehicles with occupants. In contrast, our approach targets autonomous racing, shifting the focus to the interface between a race engineer and the autonomous driving system operating the vehicle. This scope requires a responsive architecture that is more focused on accurate command understanding and low latency and less on general human-centric interaction.

Domain-specific adaptation of lightweight Small Language Models (SLMs) [15] has proven effective across a variety of specialized tasks [16]–[18]. For instance, Pal et al. [19] demonstrated that supervised fine-tuning combined with Quantized Low-Rank Adaptation (QLoRA) [20] on the Phi-3.5 Mini Instruct model [21] significantly improves intent recognition in vehicle-related tasks. This architecture addresses critical challenges such as computational constraints and latency by employing 4-bit quantization to enable efficient execution on edge hardware. Our work adopts a similar technical approach, described in detail in Section III-C.

Beyond LLM-based intent recognition, the operational viability of voice-enabled assistants in time-critical

applications is fundamentally tied to the underlying speech-to-text (STT) architecture [22], [23]. Di Leo et al. [24] propose a real-time STT framework specifically designed for edge computing environments requiring ultra-low latency and local execution. Their prototype utilizes the lightweight Vosk engine [25] for offline automatic speech recognition (ASR) [23], focusing on data privacy and operational independence by eliminating reliance on external network bandwidth. The authors highlight that their modular and extensible design is intended to accommodate the future integration of more advanced neural STT engines, such as OpenAI’s Whisper [26]. Our work adopts this local-first philosophy but directly utilizes the Whisper architecture, as described in detail in Section II-B.

II. METHOD

We integrate our *Jarvis* voice assistant with a software stack for autonomous motorsport applications [27]. We describe the integration in Section II-A and the *Jarvis* architecture itself in Section II-B.

A. System Architecture

The system is centered around a base station (Figure 1), which acts as the interface between humans and the onboard vehicle software. Moreover, it receives information about the status of the racing car and all associated software modules (Figure 2), providing telemetry, and diagnostics. The software modules include, among others, perception, planning, detection, which communicate via the Robot Operating System 2 (ROS2) [28]. Our *Jarvis* voice assistant is implemented as a stand-alone module and extends the base station by enabling a human operator to issue high-level commands using natural speech. Speech inputs are processed by the voice assistant pipeline, as described in Section II-B, where automatic speech recognition and intent interpretation generate predefined behavioral commands, such as speed targets, start/stop commands or requests to exit or return to the pit lane. These commands are transmitted to the base station using ROS2, where they undergo validation

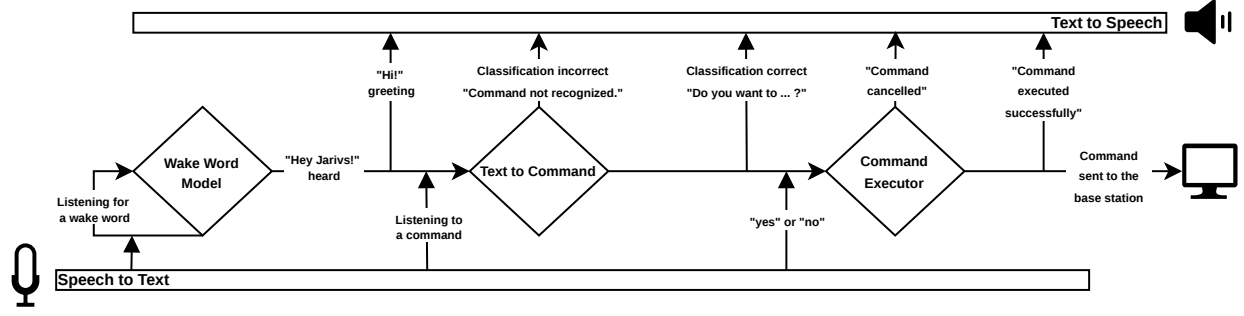


Fig. 3. System architecture and process flow of Jarvis. After wake-word detection, the system performs local speech-to-text transcription, LLM-based command classification, and text-to-speech feedback. Confirmed commands are transmitted via ROS2 to the base station, where safety checks precede transmission on the autonomous racing vehicle.

and safety checks before being accepted. Once validated, the base station forwards the commands to the racing car over a mobile network connection to the vehicle. To ensure transparency and operator awareness, every voice-enabled command is visualized in the base station interface, allowing the human controller to verify correct command detection and execution.

B. Voice Assistant Architecture

The architecture of *Jarvis* is visualized in Figure 3. The system is initiated by wake-word detection based on the openWakeWord framework¹, which continuously monitors the audio stream for the trigger phrase “Hey Jarvis!”, using keyword spotting techniques [29]. Upon activation, the assistant acknowledges the human operator with a short synthesized response “Hi!” and transitions into a command-listening state.

Speech-to-text conversion is performed locally using OpenAI’s Whisper model, specifically the lightweight English-only “base.en” variant², enabling low-latency, network-independent transcription. The resulting text is forwarded to the core LLM-based text-to-command classifier, which interprets the operator’s intent and maps the natural-language input to a predefined behavioral command. We compared online-hosted and locally deployed open-weight models for command classification. While cloud models provide strong zero-shot performance, network-based inference introduces higher latency. Fine-tuned local models achieved the best trade-off between latency and accuracy as described in Section III.

Following command interpretation, auditory feedback is generated for the operator using a text-to-speech (TTS) module based on the Coqui TTS framework³ and a VITS-based neural speech synthesis model [30]. This feedback confirms successful command recognition or indicates failure. Frequently used responses are pre-recorded to reduce latency and played back directly.

In the final step, the human operator is asked to confirm the execution of a recognized command, and only confirmed commands are transmitted to the base station via ROS2, where additional safety checks and state validations are performed before forwarding the commands to the autonomous racing car.

III. RESULTS

We conducted a multi-stage evaluation. First, we compared multiple LLMs in default configurations. We then applied two successive fine-tuning stages, resulting in the final selected model. Our primary benchmarking criteria were:

- **Intent Classification Accuracy** defined as the ratio of correctly classified commands to the total number of test samples.
- **Inference time** measured as the end-to-end processing time required for the model to classify a command. The measurement interval began the moment the tokenized input was fed into the model and ended once the final “end-of-sequence” token was generated.

A. Initial benchmarking of LLMs

We first report benchmark results for multiple LLMs evaluated without task-specific fine-tuning comprising of several online-hosted and locally deployed models in terms of inference time and initial classification accuracy. The benchmark was conducted using an expert-curated dataset of predefined high-level racing commands. The dataset of 85 samples was designed for preliminary evaluation and was paired with a prompt that described all commands and enforced a structured output format to ensure consistent command interpretation.

We evaluated OpenAI GPT-4, GPT-3.5, and o3-mini variants, listed in Table I. All models were accessed via the OpenAI⁴ API under a paid account plan, using default settings, including the temperature equal 1.0.

Additionally, we benchmarked locally deployed, open-weight SLMs from the Gamma, DeepSeek, Llama,

¹<https://github.com/dscripka/openWakeWord>

²<https://github.com/openai/whisper>

³<https://github.com/coqui-ai/TTS>

⁴<https://developers.openai.com/api/reference/overview/>

TABLE I

INITIAL BENCHMARKING OF ONLINE AND LOCAL MODELS.
HIGHLIGHTED MODELS WERE SELECTED FOR FURTHER FINE-TUNING.

Model	Accuracy [%]	Avg. Latency [s]
<i>Online Models</i>		
GPT-4o-2024-11-20	76.47	3.38
GPT-4o-2024-08-06	74.12	4.16
o3-mini-2025-01-31	85.88	21.08
GPT-4-turbo-2024-04-09	81.18	15.73
GPT-3.5-turbo-0125	57.65	5.12
<i>Locally Deployed Models</i>		
Llama3.1-8B	8.24	0.61
Llama3.2-3B	2.35	0.44
Deepseek-R1-7B	5.88	4.07
Deepseek-R1-14B	16.47	9.86
Deepseek-R1-32B	21.18	34.96
Gemma2-9B	8.24	0.82
Gemma2-27B	27.06	1.82
Gemma3-12B	18.82	1.14
Mistral-7B	27.06	0.64
Mistral-small-22B	16.47	1.31
Mistral-small-24B	24.71	1.15

and Mistral families. These experiments were run on an NVIDIA GeForce RTX 4090 GPU.

The results, summarized in Table I, reveal a trade-off between accuracy and latency. Online models generally achieved higher classification accuracy but often exhibited prohibitively long response times, making them unsuitable for our target application. In contrast, locally deployed models showed lower accuracy but significantly reduced inference latency. Given the quick response time demand of the target application, latency was prioritized over accuracy. Consequently, Mistral 7B [31], LLaMA-3.1 8B, and LLaMA-3.2 3B [32], the three models with the lowest prompt processing times, were selected for further fine-tuning. Initial accuracy deficiencies were considered secondary, as classification performance was expected to improve through fine-tuning on a domain-specific dataset.

B. Dataset Preparation for Model Fine-Tuning

The small initial dataset used for benchmarking, described in Subsection III-A was sufficient for preliminary evaluation. However, it was too small for effective fine-tuning of language models. To address this limitation, we extended the dataset. The augmentation process was performed using the GPT-4 LLM, which demonstrated strong semantic understanding in the initial benchmarking experiments. It was prompted to use various augmentation techniques [33]–[36], including synonym replacement, paraphrasing, word reordering, and minor contextual additions. These techniques increased linguistic diversity without introducing new command semantics, enabling effective fine-tuning. Beyond predefined commands, we integrated an out-of-scope (OOS) class to handle unrelated inputs. This enhances the training of the classifier to increase system robustness through more reliable rejection of irrelevant prompts. The resulting extended dataset,

comprising 17 command classes and 1,645 samples, provides increased linguistic coverage by incorporating a broader range of vocabulary and syntactic structures. It was partitioned into an 80/20 train-test split, providing a training set of 1,308 samples.

C. Fine-Tuning and Model Optimization

The three models selected in Subsection III-A for further fine-tuning, were: Mistral 7B, LLaMA-3.1 8B, and LLaMA-3.2 3B.

Training was implemented using the *Unsloth*⁵ framework to apply QLoRA, targeting all linear modules [37]. Specifically, adaptations were applied to the attention projection layers (q, k, v, o) [38] and the MLP layers ($gate, up, down$) [39].

To identify the optimal configuration, we conducted a grid search over the following discrete hyperparameter space:

- **Learning Rates:** 1×10^{-5} and 5×10^{-5}
- **Epochs:** 2 and 3
- **Gradient Accumulation Steps:** 4 and 8
- **LoRA Rank (r):** 16 and 32 (with α scaled accordingly)

Fine-tuning was performed in two stages.

D. First Stage Fine-tuning

In the first phase, we evaluated the receptiveness of all three models towards fine-tuning. Given the large number of hyperparameter combinations, and to reduce computational cost, the initial fine-tuning was conducted on a subset of five out of seventeen command classes extracted from the full training dataset (Section III-B). Accordingly, the evaluation was performed on the corresponding subset of the test dataset. All experiments in this phase were performed on an NVIDIA GeForce RTX 4090 GPU. The results are presented in Table II and identified Mistral 7B as the best-performing model, achieving the highest accuracy of 76.74 %.

E. Second Stage Fine-Tuning

Mistral 7B was selected for full fine-tuning on the complete training dataset, using the best-performing hyperparameter configuration. The full fine-tuning and evaluation was conducted on an NVIDIA GeForce RTX 3060 Laptop GPU using the full train and test data as described in Section III-B. The best-performing fine-tuned variant of Mistral-7B, configured as specified in Table III, achieved a 97.63 % classification accuracy, which exceeded the performance of every evaluated cloud-based model (Table I). Moreover, with an average inference latency of 1.39 s, it delivered faster response times, demonstrating that locally fine-tuned open-weight models can surpass the online-based ones in both accuracy and efficiency.

⁵<https://unsloth.ai>

TABLE II
INITIAL FINE-TUNING FOR BASIC COMMANDS
BEST CONFIGURATIONS FOR SELECTED MODELS

Model	Accuracy [%]	Avg. Inference [s]	Epochs	Grad Acc Steps	r	alpha
<i>Locally Deployed Models</i>						
Mistral-7B	76.74	0.92	2	4	32	32
Llama3.1-8B	75.58	0.88	3	4	32	32
Llama3.2-3B	73.26	0.67	3	4	32	32

TABLE III
BEST MODEL RESULTS AND HYPERPARAMETER AFTER SECOND STAGE
FINE-TUNING

Parameter	Value
Model Name	Mistral-7B-v0.3-bnb-4bit
Accuracy [%]	97.63
Avg. Inference [s]	1.39
Learning Rate	5×10^{-5}
Epochs	2
LoRA Rank (r)	32
LoRA Alpha (α)	32
Gradient Accumulation Steps	4
Optimizer	AdamW 8-bit

IV. DISCUSSION

The experimental results demonstrate that, for narrowly scoped command classification tasks, domain-specific fine-tuning of lightweight local language models can outperform larger online-hosted systems. While cloud-based models such as GPT-4 variants provide strong zero-shot intent and contextual understanding, they introduce an inference burden in a constrained command-classification scenario. Additionally, cloud-based models introduce network dependency, making system performance sensitive to connectivity and external service conditions. In contrast, our locally fine-tuned Mistral-7B model operates entirely on-device and is optimized for domain-specific phrasing patterns, resulting in more consistent behavior under controlled inference settings.

Our findings further highlight the effectiveness of QLoRA-based fine-tuning for edge deployment scenarios. Fine-tuning 7B-scale models using 4-bit quantization enables high task-specific performance on resource-constrained hardware, such as a mobile laptop workstation equipped with an NVIDIA GeForce RTX 3060 GPU. Our fine-tuning approach improves the accuracy of the final candidate model from 27.06 % to 97.64 %.

Despite these strengths, several limitations remain. First, although the dataset was expanded through GPT-4-based augmentation to increase linguistic diversity, it remains partially synthetic and text-based. While the augmentation process introduced paraphrastic variation, it may not capture the full distribution of possible inputs, including incomplete sentences, hesitations, or unconventional formulations. Consequently, future work should incorporate real-world operator transcripts collected under authentic racing conditions to better reflect natural language usage and

improve generalization.

Moreover, the current system operates within a strictly predefined command space. Although this constraint enhances safety and interpretability, it limits conversational flexibility. Extending the system toward more adaptive dialogue capabilities would require hierarchical command validation and formally defined safety constraints to prevent unintended behavior.

V. CONCLUSION & OUTLOOK

In this work, we presented *Jarvis*, an offline voice assistant architecture for high-level control of autonomous racing vehicles. By integrating wake-word detection, local speech-to-text transcription, and a fine-tuned lightweight language model for intent classification, we developed a system with quick response times and network-independent operation, suitable for use as a voice assistant in high-speed racing applications.

The presented architecture provides an open-source framework for an interactive voice assistant using speech-to-command classification. Beyond racing, the approach is transferable to other robotic systems where deterministic performance, low latency, and network independence are essential.

Future work includes multi-turn dialogue and bidirectional telemetry querying.

REFERENCES

- [1] J. Betz, H. Zheng, A. Liniger, U. Rosolia, P. Karle, M. Behl, V. Krovi, and R. Mangharam, "Autonomous vehicles on the edge: A survey on autonomous vehicle racing," *IEEE Open Journal of Intelligent Transportation Systems*, vol. 3, pp. 458–488, 2022.
- [2] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual instruction tuning," in *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., 2023, pp. 34 892–34 916.
- [3] C. Cui, Y. Ma, X. Cao, W. Ye, Y. Zhou, K. Liang, J. Chen, J. Lu, Z. Yang, K.-D. Liao, T. Gao, E. Li, K. Tang, Z. Cao, T. Zhou, A. Liu, X. Yan, S. Mei, J. Cao, Z. Wang, and C. Zheng, "A survey on multimodal large language models for autonomous driving," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV) Workshops*, January 2024, pp. 958–979.
- [4] J. Mao, Y. Qian, H. Zhao, and Y. Wang, "Gpt-driver: Learning to drive with gpt," *arXiv preprint arXiv:2310.01415*, 2023.
- [5] H. Sha, Y. Mu, Y. Jiang, L. Chen, C. Xu, P. Luo, S. E. Li, M. Tomizuka, W. Zhan, and M. Ding, "LanguageMPC: Large language models as decision makers for autonomous driving," *arXiv preprint arXiv:2310.03026*, 2023.
- [6] Z. Xu, Y. Zhang, E. Xie, Z. Zhao, Y. Guo, K.-Y. K. Wong, Z. Li, and H. Zhao, "DriveGPT4: Interpretable end-to-end autonomous driving via large language model," *IEEE Robotics and Automation Letters*, vol. 9, no. 10, pp. 8186–8193, 2024.

- [7] C. Sima, K. Renz, K. Chitta, L. Chen, H. Zhang, C. Xie, P. Luo, A. Geiger, and H. Li, "Drivelm: Driving with graph visual question answering," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2024, pp. 256–274.
- [8] L. Chen, O. Sinavski, J. Hünemann, A. Karnsund, A. J. Willmott, D. Birch, D. Maund, and J. Shotton, "Driving with LLMs: Fusing object-level vector modality for explainable autonomous driving," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 14 093–14 100.
- [9] B. Li, Y. Wang, J. Mao, B. Ivanovic, S. Veer, K. Leung, and M. Pavone, "Driving everywhere with large language model policy adaptation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 14 948–14 957.
- [10] Y. Yang, Q. Zhang, C. Li, D. S. o. Marta, N. Batool, and J. Folkesson, "Human-centric autonomous systems with llms for user command reasoning," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV) Workshops*, 2024, pp. 988–994.
- [11] J. He, S. Chen, F. Zhang, and Z. Yang, "From words to actions: Unveiling the theoretical underpinnings of llm-driven autonomous systems," in *Proceedings of the 41st International Conference on Machine Learning*, 2024, pp. 17 807–17 841. [Online]. Available: <https://proceedings.mlr.press/v235/he24a.html>
- [12] M. Seegert, K. Moller, and J. Betz, "Modular autonomy with conversational interaction: An llm-driven framework for decision making in autonomous driving," 2026, arXiv:2601.05806.
- [13] C. Cui, Y. Ma, S.-Y. Park, Z. Yang, Y. Zhou, P. Liu, J. Lu, J. Peng, J. Zhang, R. Zhang, L. Li, Y. Chen, J. H. Panchal, A. Abdelraouf, R. Gupta, K. Han, and Z. Wang, "Llm4ad: Large language models for autonomous driving—concept, review, benchmark, experiments, and future trends," *Proceedings of the IEEE*, pp. 1–36, 2026.
- [14] OpenAI, J. Achiam, S. Adler, and et al., "Gpt-4 technical report," 2024.
- [15] Z. Lu, X. Li, D. Cai, R. Yi, F. Liu, X. Zhang, N. D. Lane, and M. Xu, "Small language models: Survey, measurements, and insights," 2025, arXiv:2409.15790.
- [16] Y. Gu, L. Dong, F. Wei, and M. Huang, "Minillm: On-policy distillation of large language models," 2026, arXiv:2306.08543.
- [17] C.-Y. Hsieh, C.-L. Li, C.-k. Yeh, H. Nakhosht, Y. Fujii, A. Ratner, R. Krishna, C.-Y. Lee, and T. Pfister, "Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes," in *Findings of the Association for Computational Linguistics: ACL 2023*, A. Rogers, J. Boyd-Graber, and N. Okazaki, Eds., 2023, pp. 8003–8017.
- [18] S. Wang, Y. Fu, and J. Kim, "Toward construction-specialized, small language models: The interplay of domain adaptation, model scale and data volume," *Advanced Engineering Informatics*, vol. 69, p. 104035, 2026.
- [19] V. Pal, M. Kour, and G. Sethi, "Ai-powered iot assistant for electric vehicles: Fine-tuning a small language model for intent recognition and response generation," in *2025 IEEE International Transportation Electrification Conference (ITEC-India)*, 2025, pp. 1–6.
- [20] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "Qlora: Efficient finetuning of quantized llms," *Advances in Neural Information Processing Systems*, vol. 37, 2023.
- [21] M. Abidin, J. Aneja, H. Awadalla, A. Awadallah, A. A. Awan, and et al., "Phi-3 technical report: A highly capable language model locally on your phone," 2024, arXiv:2404.14219.
- [22] A. Rista and A. Kadriu, "Automatic speech recognition: A comprehensive survey," *SEEU Review*, vol. 15, no. 2, pp. 86–112, 2020. [Online]. Available: <https://doi.org/10.2478/seeur-2020-0019>
- [23] M. Nayeem, M. S. Tabrej, K. J. Deb, S. Goswami, and M. A. Hakim, "Automatic speech recognition in the modern era: Architectures, training, and evaluation," 2025, arXiv:2510.12827.
- [24] S. Di Leo, L. De Cicco, and S. Mascolo, "Real-time speech-to-text on edge: A prototype system for ultra-low latency communication with ai-powered nlp," *Information*, vol. 16, no. 8, p. 685, 2025.
- [25] A. A. Soni, "Improving speech recognition accuracy using custom language models with the vosk toolkit," 2025, arXiv:2503.21025.
- [26] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proceedings of the 40th International Conference on Machine Learning*, ser. ICML'23, 2023.
- [27] S. Hoffmann, S. Sagmeister, T. Betz, J. Bongard, S. Büttner, D. Ebner, D. Esser, G. Jank, S. Goblirsch, A. Langmann, M. Leitenstern, L. Ögretmen, P. Pitschi, A.-K. Schwehn, C. Schröder, M. Weinmann, F. Werner, B. Lohmann, J. Betz, and M. Lienkamp, "Head-to-head autonomous racing at the limits of handling in the a2rl challenge," 2026. [Online]. Available: <https://arxiv.org/abs/2602.08571>
- [28] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Ng, "Ros: an open-source robot operating system," in *ICRA Workshop on Open Source Software*, 2009.
- [29] G. Chen, C. Parada, and G. Heigold, "Small-footprint keyword spotting using deep neural networks," in *2014 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2014, pp. 4087–4091.
- [30] J. Kim, J. Kong, and J. Son, "Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech," in *Proceedings of the 38th International Conference on Machine Learning*, vol. 139, 2021, pp. 5530–5540.
- [31] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. L. Scao, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed, "Mistral 7b," 2023, arXiv:2310.06825.
- [32] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, and et al., "The llama 3 herd of models," 2024, arXiv:2407.21783.
- [33] Z. Wang, P. Wang, K. Liu, P. Wang, Y. Fu, C.-T. Lu, C. C. Aggarwal, J. Pei, and Y. Zhou, "A comprehensive survey on data augmentation," *IEEE Transactions on Knowledge and Data Engineering*, vol. 38, no. 1, pp. 47–66, 2026.
- [34] F. Gao, J. Zhu, L. Wu, Y. Xia, T. Qin, X. Cheng, W. Zhou, and T.-Y. Liu, "Soft contextual data augmentation for neural machine translation," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019, pp. 5539–5544.
- [35] A. Karimi, L. Rossi, and A. Prati, "AEDA: An easier data augmentation technique for text classification," in *Findings of the Association for Computational Linguistics: EMNLP 2021*, 2021, pp. 2748–2754.
- [36] J. Wei and K. Zou, "EDA: Easy data augmentation techniques for boosting performance on text classification tasks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019, pp. 6382–6388.
- [37] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "Lora: Low-rank adaptation of large language models," 2021, arXiv:2106.09685.
- [38] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [39] N. Shazeer, "Glu variants improve transformer," 2020, arXiv:2002.05202.